

Notes:

- HW1 due Today
- HW2 due October 16

Last class: More git

- gitignore files
- remote repos and GitHub
- push, pull, merging
- conflicts and git stash
- Started talking about good code

Good code should be:

- readable
- runnable
- correct
- documented
- open

What makes code runnable?

- All data needed to run the code must be recorded!
 - command line arguments, input data files, parameter values, ...
 - **EVERYTHING!**
- Workflows should be automated as much as possible. What intermediate products and final results are generated and how are they used?

Mindset: Should be able to reproduce all data with well-documented commands provided in scripts

This is where Jupyter notebooks are far from optimal. Results in a notebook depend on execution order, difficult to record histories...

Imagine that you have 4 cells: Run

Cell A --> Cell B --> Cell C

Suppose I run Cell A to produce some data for Cell B which produces data and a graph in Cell C. Then I reset a parameter in Cell A to produce data in another cell:

Cell A --> Cell D

Can I reproduce Cell C from the current state of the notebook? Usually not!

Because we cannot always reproduce results within a notebook, they make a poor choice for archival versions of code. You can develop in a notebook, but should be sure to run final versions of code in Python scripts that always execute in the same order

Your job doesn't end when the code runs correctly once! Must be able to reproduce results

Dependencies must be documented carefully!

- What libraries did you use and what versions of those libraries?

For Python projects, use `requirements.txt` or `pyproject.toml` file to specify libraries/modules that were used, including versions.

Use `pip freeze` (`pip3 freeze`) to list installed packages and their versions

```
math-scott02 (~)> pip3 freeze
appnope==0.1.4
asttokens==3.0.0
certifi==2025.4.26
comm==0.2.2
contourpy==1.3.2
cycler==0.12.1
debugpy==1.8.9
decorator==5.1.1
```

requirements.txt:

```
Cython>=3.0
libsupermesh
mpi4py>3; python_version >= '3.13'
mpi4py; python_version < '3.13'
numpy
pkgconfig
pybind11
setuptools>61.2
rtree>=1.2
```

pyproject.toml:

```
dependencies = [
    "cachetools",
    "decorator<=4.4.2",
    "mpi4py>3; python_version >= '3.13'",
    "mpi4py; python_version < '3.13'",
]
```

Write `requirements.txt` (or `pyproject.toml`) with one library per line, using `=` to force specific version number or inequalities to allow a range, use semicolons to give conditions on which version

For more complicated dependencies, use zenodo or similar tools.

Zenodo is an open data archive run by CERN that stores large datasets and software, including metadata that can track dependencies. Can use this to cite specific instances of software that were used to present data. Many software libraries that use zenodo give special installation commands to install specific versions as archived by zenodo.

Docker/virtualization is also a tool, encapsulates full computational environment. Improves reproducibility (so long as you can run the virtual environment), at expense of performance.

How do we ensure code is correct?

- work openly, contribute code to public repositories so that others can look at it!
- use "unit testing". For each function in your code, prepare simple tests to validate that your function does what you think it should. A good library is one that comes with a comprehensive set of unit tests!
- We will discuss unit testing and how we integrate it into the development process next week.

- Learn to use a debugger! When errors in code are identified, step through the function with a known input-output pair and see where internal states differ from expected behaviour. More on this in a few weeks...

- "good bugs" become good test cases!

Collaborative version control is an excellent way to improve code correctness.

Basic philosophy is that all code should be looked at by at least 2 people. The person who wrote it and an independent reviewer.

In git, we manage this using *branching* and pull requests.

```
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git checkout -b scottmac/add_feature
Switched to a new branch 'scottmac/add_feature'
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> nano fancy_code.py
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git add fancy_code.py
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git commit -m"Adding fancy code"
[scottmac/add_feature f3ac498] Adding fancy code
1 file changed, 3 insertions(+)
 create mode 100644 fancy_code.py
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git push
fatal: The current branch scottmac/add_feature has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin scottmac/add_feature
```

To have this happen automatically for branches without a tracking upstream, see 'push.autoSetupRemote' in 'git help config'.

```
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git push --set-upstream origin scottmac/add_feature
Enumerating objects: 78, done.
Counting objects: 100% (78/78), done.
Delta compression using up to 10 threads
Compressing objects: 100% (53/53), done.
Writing objects: 100% (78/78), 6.54 KiB | 6.54 MiB/s, done.
Total 78 (delta 24), reused 37 (delta 3), pack-reused 0
remote: Resolving deltas: 100% (24/24), done.
remote:
remote: Create a pull request for 'scottmac/add_feature' on GitHub by visiting:
remote:   https://github.com/MacLachlanTeaching/CMSC6950_Fall2025_example/pull/new/scottmac/add_feature
remote:
To github.com:MacLachlanTeaching/CMSC6950_Fall2025_example.git
 * [new branch]      scottmac/add_feature -> scottmac/add_feature
branch 'scottmac/add_feature' set up to track 'origin/scottmac/add_feature'.
```

Now says "create a pull request"

What happened?

Go to github and see what it says:

Main branch of repo hasn't changed, now there's a second branch "1 commit ahead"

Hit "Compare and Pull Request" button, fill out form

Create a Pull Request, which is how git tracks code that we want to add to another branch. Usual process is to review code in the PR, once that code is reviewed and passes any existing/new tests, then PR can be merged into the main (or another) branch.

Review the code, make comments. Then go back and address them:

```
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> nano fancy_code.py
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git pull
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 4 (delta 2), reused 4 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 346 bytes | 69.00 KiB/s, done.
From github.com:MacLachlanTeaching/CMSC6950_Fall2025_example
 b6afaa6..3f2d7b6  main       -> origin/main
Already up to date.
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git add fancy_code.py
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git commit -m"Changes from code review"
[scottmac/add_feature 9c6f398] Changes from code review
1 file changed, 2 insertions(+), 2 deletions(-)
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 322 bytes | 322.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:MacLachlanTeaching/CMSC6950_Fall2025_example
 f3ac498..9c6f398  scottmac/add_feature -> scottmac/add_feature
```

Back to my PR.

- normally, there are specific rules about reviews needed to merge, passing tests, style, and so forth.
- here, hit "merge PR" button. Confirmed the merge, edited the commit message, deleted the branch. No if I go back to "code" tab on github, see `fancy_code.py` is there.

```
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (2/2), 994 bytes | 331.00 KiB/s, done.
From github.com:MacLachlanTeaching/CMSC6950_Fall2025_example
 3f2d7b6..3ad1807  main       -> origin/main
Your configuration specifies to merge with the ref 'refs/heads/scottmac/add_feature'
from the remote, but no such ref was fetched.
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git checkout main
Switched to branch 'main'
Your branch is behind 'origin/main' by 4 commits, and can be fast-forwarded.
( use "git pull" to update your local branch)
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git pull
Updating b6afaa6..3ad1807
Fast-forward
 CMSC6950/schedule.txt | 1 +
 fancy_code.py         | 3 +++
2 files changed, 4 insertions(+)
 create mode 100644 fancy_code.py
math-scott02 (~...Fall2025/CMSC6950_Fall2025_example)> git log --oneline -8
3ad1807 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from MacLachlanTeaching/scottmac/add_feature
9c6f398 (origin/scottmac/add_feature, scottmac/add_feature) Changes from code review
f3ac498 Adding fancy code
3f2d7b6 Adding HW6 to schedule
b6afaa6 Fixing merge, keeping both changes
a3cfcbe Adding HW5 to schedule
09b26ee Merge branch 'main' of github.com:MacLachlanTeaching/CMSC6950_Fall2025_example
00d462a Adding HW4 due date
```

Pull Requests are very effective in combination with using an issue tracker (also on GitHub)

Create issues for complex tasks (new features, bug fixes), create a branch to work on issue, and PRs to merge changes related to this back to main branch.

Similar features with "Merge Requests", based on a "forked" copies of a repository instead of branches

What makes good documentation?

- Document interfaces! Tell us what the inputs should be for each function and what the outputs are. Comments should explain reasons why a code is written in a certain way, not just repeat implementation.
- If code is very complicated and needs lots of explanations for a user to understand it, rewrite the code!

The best way to document software is to embed the documentation in the code itself, so that we update the documentation while we update the code.

Documentation generators, such as Doxygen or sphinx, translate code comments into well-formatted documentation. Use these tools!

We put a lot of effort into writing code. Don't waste it!

Code that is "tempermental", undocumented, or unrunnable is not useful code. It can never be trusted code.

If your code is not *open*, meaning that others are not free to look at, use, and extend your code, it won't be *trusted* for scientific practice.

Do not believe the myth that your work will be stolen if you don't hide it. Write code to produce manuscripts. Assert your authorship and ownership by making code available simultaneously with scientific manuscripts. (E.g., on a preprint server like the arXiv)

Note that none of the above touches on performance. It is more important that your code be correct than it be fast.